Week 6 – Design Studio

Design the airline's disruption assistant

The system. A storm closes the airline's largest hub for the evening. Hundreds of flights are cancelled or delayed, and every passenger on them opens the app and asks the same thing in their own words: what can I do? The assistant answers from the airline's own documents, with the document it relied on, for this passenger and this flight, as things stand right now.

- **Input:** the passenger (signed in, with a booking on a flight today), their question, and whatever else you decide the system can know at that moment
- **Output:** a paragraph: what this passenger can do now, with the rule or waiver it comes from quoted and linked; or a handoff to a person
- **Decision:** none by the model. The passenger rebooks, claims, or waits; an agent acts on what the assistant could not answer.
- **Scale:** 1,500 flights a day to 160 airports; 200,000 passengers flying today. 20,000 questions on an ordinary day; 300,000 on a storm evening, 150 a second at the peak, and the peak arrives in the same hour the documents change fastest.
- **The passenger:** an answer within 5 seconds, on a phone, at a gate.
- **Ground truth:** a rebooking completed in the session, minutes later; a refund claim accepted or denied, days later; a complaint, weeks later. For most answers, nothing.

What the assistant answers from. Three kinds of document, and they change at three different rates:

SOURCE	WHAT IT IS	HOW IT CHANGES
the fare rules	the contract of carriage, the fare conditions, the rebooking and refund policies: about 2,000 documents	monthly, by the policy team, through review
the waivers	during a disruption, operations publishes a waiver: which routes and dates, what the passenger may do (rebook free, refund, a hotel), until when. Five to forty on a storm day	by the hour: published, extended, withdrawn, each with an expiry
the flight status	every flight's status, gate, new departure time, and the rebooking options the operations system has already found for it	by the minute, several times an hour per flight during a storm

And one thing that is nobody else's business: **the passenger's own booking**: their flight, fare class, what they paid, what they have already been offered.

The numbers:

NUMBER	VALUE
questions at the peak	**150 a second**, in the same hour the waivers change fastest
the answer	a paragraph with the waiver quoted, **within 5 s**
the fare rules	**2,000 documents**, changing monthly
the waivers	**5 to 40 a day** during a disruption, each with routes, dates and an expiry; changed or withdrawn within the hour
the flight status	**1,500 flights**, each changing several times an hour
the vendor	**50 generations a second** contracted; 300 ms to embed a question; 2 to 4 s to generate
unanswerable	**about a third** of questions: compensation the rules do not give, a promise an agent made by phone, a hotel the airline does not offer tonight

Task.

1. **Architecture.** What is computed about every document before any question, what is computed every few minutes, and what is computed at the request? Draw the whole system: the services, what is orchestrated, the stores, and the flow between them. Show how each source reaches an answer: a fare rule, a waiver, a flight-status update, this passenger's booking. Be ready to say, for each, how old it can be by the time the passenger reads it.
2. **Data handling.** The tables, one row each. A waiver as a row, exactly, with its expiry. The labelled set: what a labelled question is pinned to, and what happens to it when the waiver it depends on is withdrawn. Trace one waiver, published at 18:05, to the first answer that quotes it, and to the first answer that stops quoting it after it is withdrawn at 21:30.
3. **Measurement.**
   - Per stage, before any passenger sees it: what is measured, and against what
   - The unanswerable slice: how it is built, and what it measures
   - With passengers: what decides a release
   - One guardrail that can stop a release whose primary metric rose

Your design should handle:

1. The three sources at their three rates. For each: is it indexed, precomputed, or looked up at the request? When it changes, how does the change reach the next answer, and how long can that take?

What as-of does the answer carry for it?

2. The passenger's own booking: where it enters the context, and why it never enters the shared index
3. A waiver as a document: how it is chunked, what is embedded, what rides as metadata, and what happens to an answer the minute the waiver expires
4. A disruption briefing in every prompt: what it says, who builds it, how often, how it is checked, and how its as-of shows up in the answer
5. A router that decides what the question needs (a rule, a status, or both) from a handful of examples in the prompt: where the examples live, and how they are tested
6. A cache, because ten thousand passengers ask the same question in the same hour: what the key is, and what makes an entry wrong
7. The quote rule: anything that commits the airline to money (a refund, a hotel, a free change) is quoted verbatim from the rule or waiver with its id, never paraphrased. Say what checks it, on every answer
8. The floor and the handoff: when the assistant does not answer, what the passenger sees, and what the agent who picks it up receives
9. A follow-up question ("and what about my bag?"): what is carried from the first answer, what is done again, and where the conversation lives

(Bonus) the vendor's rate limit arrives with the storm: 150 questions a second against 50 generations a second. Design the degraded mode: what the passenger still gets, and what the system never does.

One question to answer somewhere in your design. At 19:40 the assistant told a passenger their hotel tonight would be covered, and quoted the waiver. The waiver had been withdrawn at 19:30. Say what the log holds for that answer, which component let it through, and which component should have stopped it.